29 jun 2023

① GREP that takes groups of 2 words separated by a single space, formed by lowercase letters and each word contains at least 2 vowels.

✓ grep -E '[bcd....yz]*[aeiou][bcd...yz]*[aeiou][bcd...yz]* [b...z]*[aeiou][b...z]*[a-u][b...

space

... filename

only the text that matches

consonants vowel consonants (0 or >) (0 or >) — vowel consonants

cons vow cons (0 or >) (0 or >) — vow cons

guarantees 2 vowels.

② 2 GREP that take lines not having nr. of characters : 3.

✓ ✓ 1) grep -E -v "^(...)*$" filename

-v → invert (NOT match)

^ → start line

(...) → exactly 3 characters

$ → end line

✓ 2) grep -E -v "^(.{3})*$" filename

(.{3}) → exactly 3 characters

③ SED replacing first occurence of A with B.

✓ ✓ sed 's/A/B/'

s → substitute; A → old; B → new

✓ ✓ * replace ALL occurences:

sed 's/A/B/g'

s → substitute; g → all occurences

④ AWK displaying lines having { first & word = last word
penultimate word has even nr. of characters

✓

awk 'NF>1 && $1==$NF && length($(NF-1))%2==0 {print $0}'

identical

$1 → first word (value)

$NF → last word (value)

nr. of characters is even

length($(NF-1)) → value of penultimate field

$0 → full line (current)

⑤ 3 methods of creating an empty file.

✓ 1) touch file

2) > file

3) echo -n > file

~ 4) nano file

⑥ 5 methods of checking if a string is empty

1) test -z "$string"

↳ "zero-length" string

2) test "$string" == ""

3) ! test -n "$string"

↳ "non-zero" length

✓ 4) test ${#string} -eq 0

length → equal → 0

5) test ${#string} -lt 1

length → less than

⑦ Process hierarchy. ?

```
for(i=0;i<3;i++)
    if(fork()!=0)
        wait();
```

P (i=0)
↳ C1
↳ C2 (i=1)
↳ C3 (i=2)

⇓

parent cannot create second child until C1 finishes. But when C1 finishes, we ran out of iterations.

⑧ What will display:

```
execlp("expr","expr","a","+","1")
printf("xyz\n");
```

⇒ error because a + 1 is invalid arithmetic ⇒ expr receives: non-integer arg.

⇒ even if ~~execlp succeeded~~, the printf line would not be reached.

↓

execlp actually succeeded because it finds and launches the "expr" ⇒

⇒ this action overwrites the current process with the new program

⇒ the new program is the one that crashes due to a math error, but even if it was no error, printf would never be reached.

2

Implement popen, pclose. (see 22 june '23)

popen:
- create pipe → handle data routing
- fork process → splits execution so that child can redirect its stdin/output to the pipe
  - child
    - dup2
    - exec
  - parent
    - fdopen → closes unused pipeline and maps the remaining file descriptor via fdopen.

pclose
- fclose
- waitpid
- return status

⑩ How many FIFOs can a FILE open if each of those FIFOs will have their end open to another process?

✓ => Other process opens opposite end. => open succeeds.

- It depends on the file descriptor limits of the system (check with "ulimits -n")
- Important to open correctly and in the same order those FIFOS, otherwise it would lead to deadlocks. If correctly opened, limit is the file descriptor limit of the system.

⑪ execl vs execv

execlp
‖
replace current running process with a new program.
↓
L = list
‖
when you have fixed, known number of arguments.

execl("ls", "ls", "-l", NULL)

execvp
‖
search path + execute new program.
→ V = vector
‖
when ~~you have~~ the number of arguments is DYNAMIC or determined at runtime.

```
char *v[] = {"ls", "-l", NULL};
execv("/bin/ls", v);
```

⑫ Critical section (see 22 june 2023)

⑬ - ?

⑭ pthread_mutex_lock ⊄⊃ sem_post

pthread_mutex_lock
‖
enter protected section (one thread)

sem_post
‖
increase semaphore release permission.

The ~~thread~~ counter decreases on sem_wait as a thread enters the critical section, and when the counter reaches 0, any other threads that did not fit are blocked from entering. When a thread finishes, it must explicitly call sem_post to leave. Calling sem_post increases the counter signaling that a thread has left and allowing a blocked thread to enter. ③

Conclusion: replacing pthread_mutex_lock with sem_post leads to chaos, since sem_post increments the semaphore, allowing any thread to enter the critical section, thus failing to prevent concurrent access to shared resources. =>
=> race conditions, data corruption.

(15) Binary semaphore
= sync. mechanism that can have only two values: 0 and 1
Operations: → (wait) → decrements the semaphore (from 1 to 0)
↑ post → (signal) → increments the semaphore (from 0 to 1)

(16) -

(17) prevent deadlocks: - consistent locking order
- avoid nested loops
- deadlock detection algorithms
- lock timeouts

(18) pthread_join → process status?

running state → wait (block) → ready → running

thread calls pthread_join

Initial state: Thread is in the running state and calls pthread_join.
Waiting for termination: Thread calling pthread_join moves to the blocked state, waiting for the target thread to finish.
Target thread terminates: Once target thread finishes, it changes its state to terminated.
Unblocking: The waiting thread is unblocked and moves to the ready state.
Resuming execution: the thread is eventually scheduled by the OS and returns to the running state.

(19) B = block size; A = address size; How many addresses will a double indirect have in an i-node?
$\left(\frac{B}{A}\right)^2$ because first block points to B/A blocks, each of those points to B/A blocks

(20) What happens with the contents of the directory in which we mount a partition /mnt (initially) ← files
mount → partition contents become visible and original files are hidden.?
The existing content of the directory where we are mounting becomes temporarily inaccessible. After mounting, any access to the directory actually accesses the root of the mounted partition.